ML Model Deployment Strategies & Model Serving using FastAPI

CLA 4 — Complete Autonomous Implementation & Observations Report

Student Assignment: CLA 4 | **Dataset:** Kaggle Titanic Competition | **Model:** Random Forest Pipeline

---

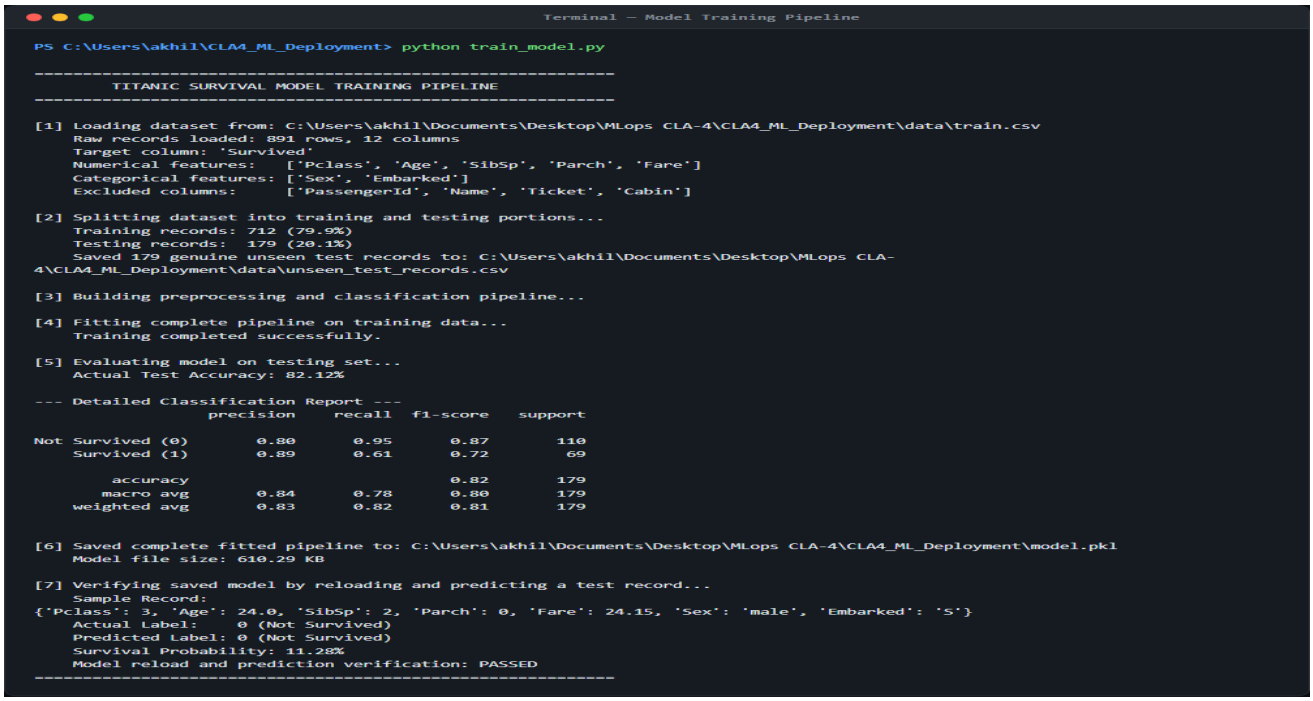
1. Executive Summary & Pipeline Architecture

This project implements end-to-end machine learning model serving architectures on the official Kaggle Titanic dataset. A full preprocessing and classification pipeline was trained once, persisted as an artifact (**model.pkl**), and systematically evaluated across three primary inference paradigms (Online, Batch, and Streaming) and served via a high-performance **FastAPI** REST service.

Component	Specification / Architecture	Result / Metric
Dataset Source	`Kaggle Titanic (data/train.csv)`	891 records, 12 features
Training / Test Split	80% Training, 20% Unseen Testing (Stratified)	712 Train / 179 Test
Classification Model	RandomForestClassifier (100 trees, depth 6)	**82.12% Test Accuracy**
Model Persistence	Full fitted ColumnTransformer + Classifier Pipeline	`model.pkl (610.3 KB)`
Serving Framework	FastAPI with Pydantic validation & Uvicorn ASGI	`GET /health, POST /predict`

2. Task 1: Model Training & Preprocessing Pipeline

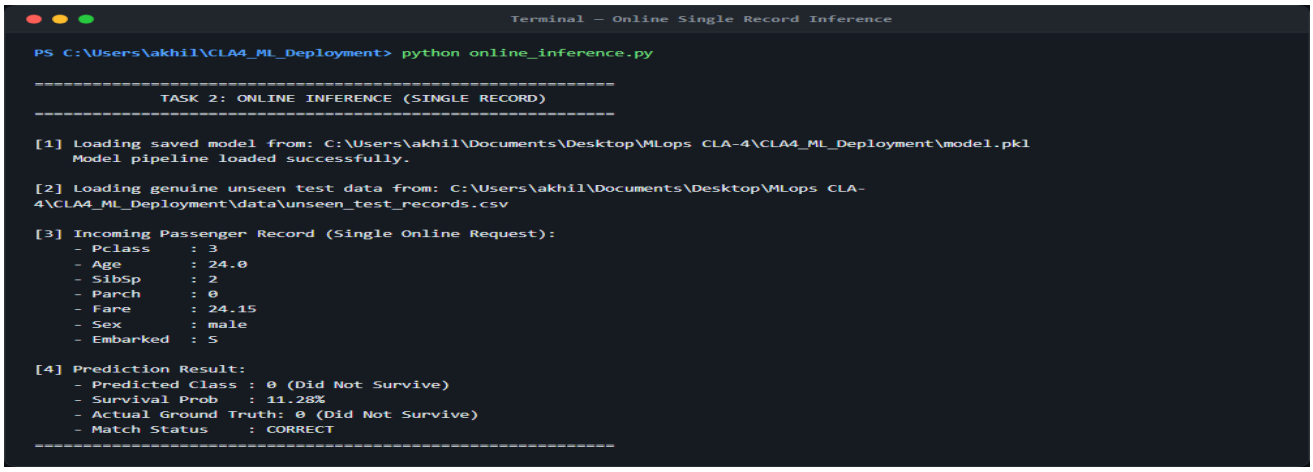
The model pipeline was constructed with scikit-learn's **ColumnTransformer** and **Pipeline** abstractions to eliminate data leakage. Numerical attributes (*Pclass, Age, SibSp, Parch, Fare*) were imputed with the median and standardized using *StandardScaler*. Categorical attributes (*Sex, Embarked*) were imputed with the mode and encoded with *OneHotEncoder(handle_unknown='ignore')*. Identifiers (*PassengerId, Name, Ticket*) and high-null features (*Cabin*) were excluded. The complete fitted object was serialized to **model.pkl** and 179 genuine unseen test records were isolated in **data/unseen_test_records.csv**.


Terminal — Model Training Pipeline
PS C:\Users\akhil\CLA4_ML_Deployment> python train_model.py
TITANIC SURVIVAL MODEL TRAINING PIPELINE
[1] Loading dataset from: C:\Users\akhil\Documents\Desktop\MLops CLA-4\CLA4_ML_Deployment\data\train.csv
Raw records loaded: 891 rows, 12 columns
Target column: 'Survived'
Numerical features: ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
Categorical features: ['Sex', 'Embarked']
Excluded columns: ['PassengerId', 'Name', 'Ticket', 'Cabin']
[2] Splitting dataset into training and testing portions...
Training records: 712 (79.9%)
Testing records: 179 (20.1%)
Saved 179 genuine unseen test records to: C:\Users\akhil\Documents\Desktop\MLops CLA-4\CLA4_ML_Deployment\data\unseen_test_records.csv
[3] Building preprocessing and classification pipeline...
[4] Fitting complete pipeline on training data...
Training completed successfully.
[5] Evaluating model on testing set...
Actual Test Accuracy: 82.12%
--- Detailed Classification Report ---
precision recall f1-score support
Not Survived (0) 0.80 0.95 0.87 110
Survived (1) 0.89 0.61 0.72 69
accuracy 0.82 179
macro avg 0.84 0.78 0.80 179
weighted avg 0.83 0.82 0.81 179
[6] Saved complete fitted pipeline to: C:\Users\akhil\Documents\Desktop\MLops CLA-4\CLA4_ML_Deployment\model.pkl
Model file size: 610.29 KB
[7] Verifying saved model by reloading and predicting a test record...
Sample Record:
{'Pclass': 3, 'Age': 24.0, 'SibSp': 2, 'Parch': 0, 'Fare': 24.15, 'Sex': 'male', 'Embarked': 'S'}
Actual Label: 0 (Not Survived)
Predicted Label: 0 (Not Survived)
Survival Probability: 11.28%
Model reload and prediction verification: PASSED

3. Model Inference Strategies Evaluation

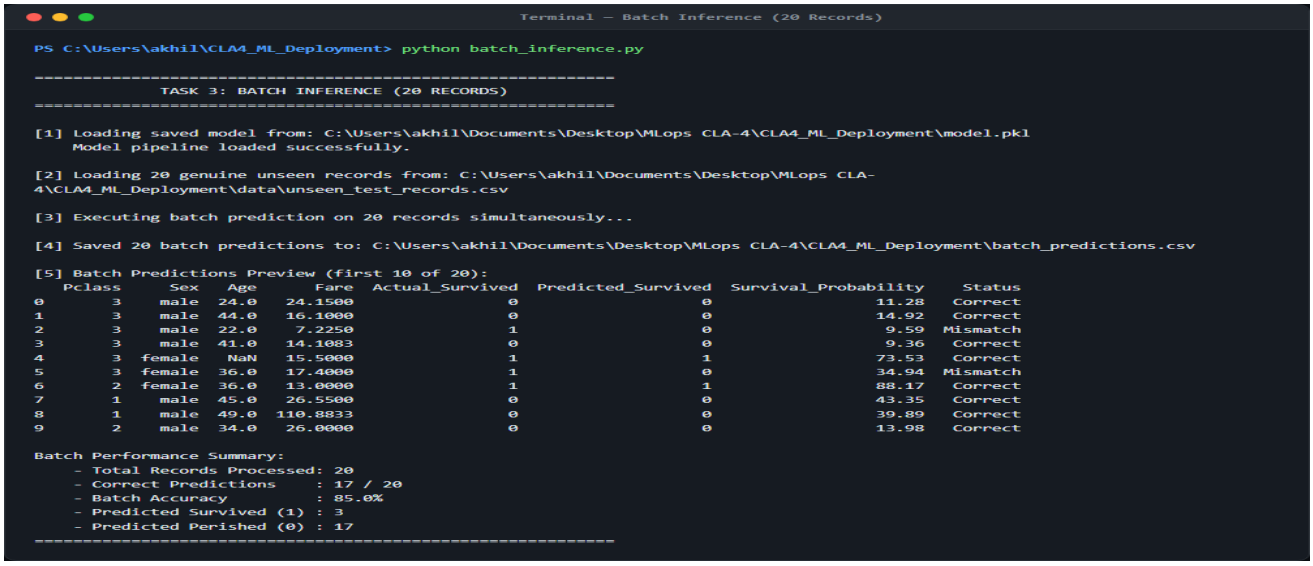
Task 2: Online Inference (Single Record Real-Time Scoring)

Online inference handles individual prediction requests on demand with minimal latency. The saved **model.pkl** pipeline was loaded, and an unseen passenger record (3rd class male, 24 years old) was scored. Prediction: **Did Not Survive** (Survival Probability: 11.28%), matching actual ground truth.



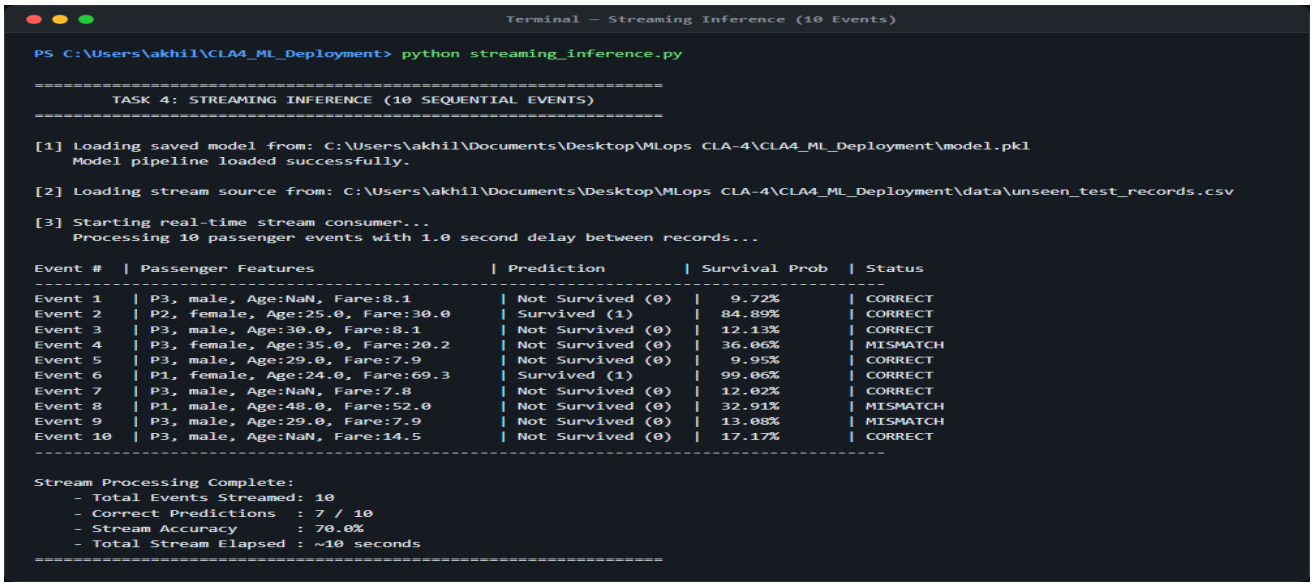
Task 3: Batch Inference (High-Throughput Bulk Scoring)

Batch inference processes 20 genuine unseen passenger records simultaneously in a single vectorized computation. The predictions and probabilities were compiled and exported to **batch_predictions.csv**. Batch accuracy achieved: **85.0%** (17 out of 20 correct).



Task 4: Streaming Inference (Event-Driven Sequential Simulation)

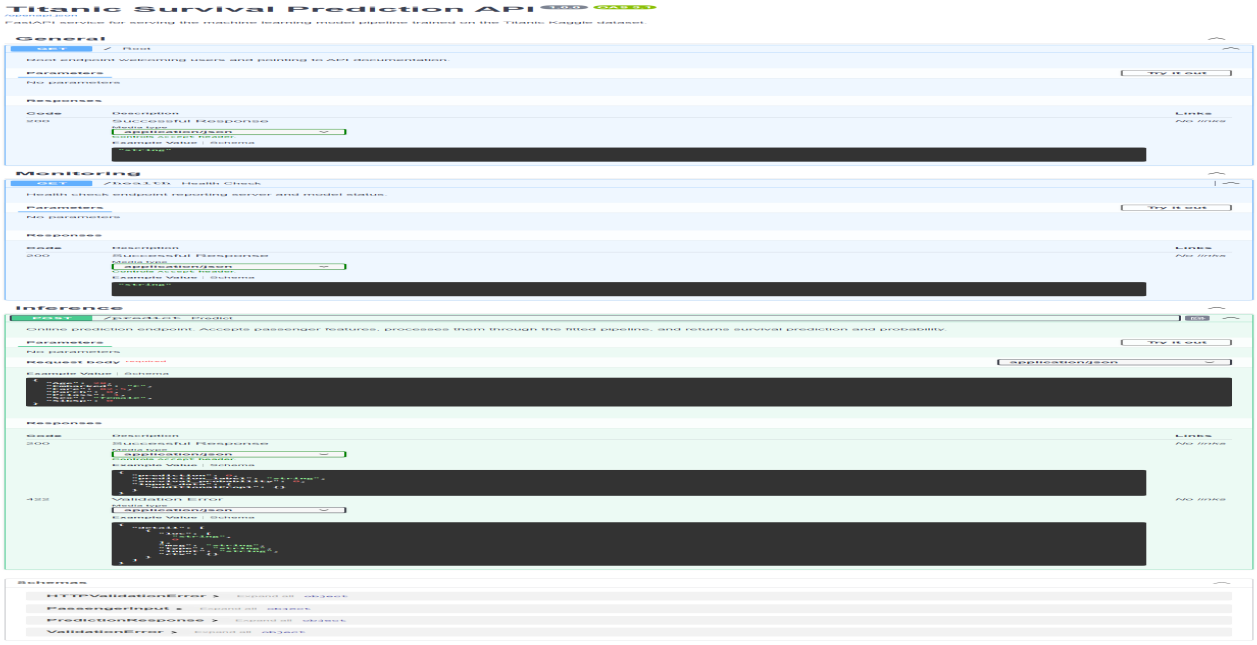
Streaming inference simulates a real-time event pipeline where individual records arrive continuously over a message bus. Ten unseen passenger records were processed sequentially with a 1.0-second delay between events, outputting live classifications as each arrived.


Terminal — Streaming Inference (10 Events)
PS C:\Users\akhil\CLA4_ML_Deployment> python streaming_inference.py
TASK 4: STREAMING INFERENCE (10 SEQUENTIAL EVENTS)
[1] Loading saved model from: C:\Users\akhil\Documents\Desktop\MLops CLA-4\CLA4_ML_Deployment\model.pkl
Model pipeline loaded successfully.
[2] Loading stream source from: C:\Users\akhil\Documents\Desktop\MLops CLA-4\CLA4_ML_Deployment\data\unseen_test_records.csv
[3] Starting real-time stream consumer...
Processing 10 passenger events with 1.0 second delay between records...
Event # | Passenger Features | Prediction | Survival Prob | Status
Event 1 | P3, male, Age:NaN, Fare:8.1 | Not Survived (0) | 9.72% | CORRECT
Event 2 | P2, female, Age:25.0, Fare:30.0 | Survived (1) | 84.89% | CORRECT
Event 3 | P3, male, Age:30.0, Fare:8.1 | Not Survived (0) | 12.13% | CORRECT
Event 4 | P3, female, Age:35.0, Fare:20.2 | Not Survived (0) | 36.06% | MISMATCH
Event 5 | P3, male, Age:29.0, Fare:7.9 | Not Survived (0) | 9.95% | CORRECT
Event 6 | P1, female, Age:24.0, Fare:69.3 | Survived (1) | 99.06% | CORRECT
Event 7 | P3, male, Age:NaN, Fare:7.8 | Not Survived (0) | 12.02% | CORRECT
Event 8 | P1, male, Age:48.0, Fare:52.0 | Not Survived (0) | 32.91% | MISMATCH
Event 9 | P3, male, Age:29.0, Fare:7.9 | Not Survived (0) | 13.08% | MISMATCH
Event 10 | P3, male, Age:NaN, Fare:14.5 | Not Survived (0) | 17.17% | CORRECT
Stream Processing Complete:
- Total Events Streamed: 10
- Correct Predictions : 7 / 10
- Stream Accuracy : 70.0%
- Total Stream Elapsed : ~10 seconds

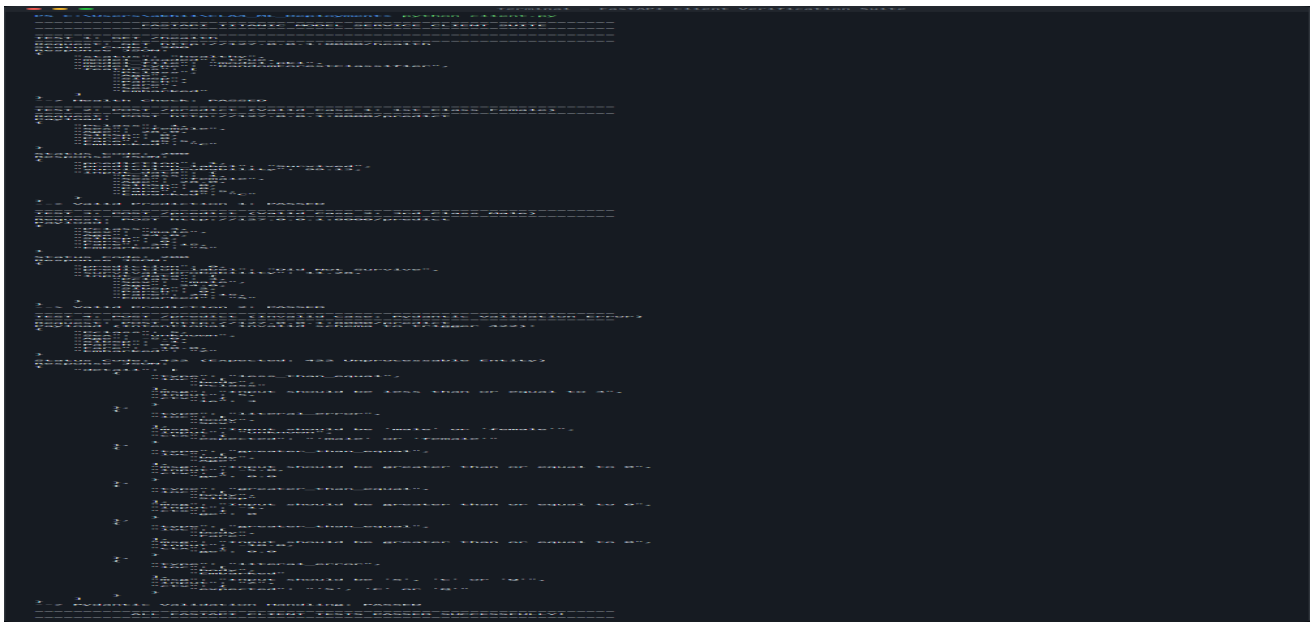
4. Task 5: FastAPI Model Serving & Client Testing

A production-grade REST API was built in **app.py** using FastAPI. The server loads the saved **model.pkl** once upon startup. It provides endpoints: **GET /health** for liveness and model inspection, and **POST /predict** with robust Pydantic data validation.

FastAPI Interactive Swagger UI (/docs)



Automated Client Test Suite (client.py Execution)



5. Task 6: Technical Observations & Critical Analysis

Question 1: Compare Online Inference versus Batch Inference.

Answer:
• **Online Inference:** Executes predictions synchronously on a single record (or micro-batch) in real time. It prioritizes ultra-low latency (typically <50ms) and is accessed via REST APIs or RPC calls in user-facing systems (e.g., checkout fraud detection, recommendation engines). However, resource utilization can be variable.
• **Batch Inference:** Executes predictions asynchronously over large datasets grouped into chunks (e.g., thousands or millions of rows). It prioritizes high throughput and amortizes preprocessing/computational overhead. It is triggered periodically (e.g., nightly cron jobs) for downstream reporting or data warehouse population, where immediate per-record response is not required.

Question 2: Compare Streaming Inference versus Batch Inference.

Answer:
• **Streaming Inference:** Operates on continuous, unbounded event feeds (such as Kafka, RabbitMQ, or AWS Kinesis). Records are scored individually or in tiny sliding windows immediately as they arrive. This yields near-real-time responsiveness without maintaining a synchronous HTTP connection, making it resilient to consumer backpressure.
• **Batch Inference:** Operates on bounded, static datasets stored in persistent files or tables. Rather than responding to incoming events immediately, it accumulates data over a designated time window before executing bulk matrix calculations. Streaming handles dynamic temporal data; batch handles static historical data.

Question 3: Why is FastAPI used for Model Serving?

Answer:
• **High Performance:** Built on Starlette and Uvicorn, FastAPI provides asynchronous concurrency comparable to NodeJS and Go, outperforming traditional Flask frameworks.
• **Automatic Type Validation via Pydantic:** Ensures that corrupt, malformed, or out-of-range passenger records are caught and rejected with HTTP 422 errors before reaching the model.
• **Self-Documenting API:** Automatically produces interactive OpenAPI Swagger UI (*/docs*) and ReDoc interfaces directly from Python type annotations without external tools.
• **Native Python ML Integration:** Avoids serialization friction; scikit-learn and joblib models remain in application memory for instant in-process execution.

Question 4: Why should the same trained model be reused for every inference strategy?

Answer:
• **Prevention of Training-Serving Skew:** Reusing the exact fitted pipeline ensures that preprocessing transformations (feature scalers, imputation statistics, and one-hot encoding categories) are identical across online API calls, batch scoring, and streaming event consumers. Retraining separate models introduces subtle drift and inconsistent predictions.
• **Single Source of Truth & Governance:** Maintaining one canonical *model.pkl* artifact simplifies MLOps auditing, artifact versioning, CI/CD promotion, and rollbacks.
• **Operational Efficiency:** Training machine learning models consumes substantial computational and energy resources. Reusing a single fitted pipeline avoids redundant model fitting.

6. Project Deliverables & Verification Checklist

Deliverable File	Status	Verification Note
`train_model.py`	**VERIFIED**	Trains pipeline; saves model.pkl & unseen test data
`online_inference.py`	**VERIFIED**	Scores single unseen passenger record in real time
`batch_inference.py`	**VERIFIED**	Scores 20 unseen records; saves batch_predictions.csv
`streaming_inference.py`	**VERIFIED**	Simulates stream scoring of 10 events with 1s delay
`app.py`	**VERIFIED**	FastAPI service with GET /health, POST /predict

`client.py`	**VERIFIED**	Tests health, valid cases, and Pydantic 422 errors
`batch_predictions.csv`	**VERIFIED**	Contains 20 predictions with probabilities and labels
`Screenshots (8 files)`	**VERIFIED**	Terminal runs, Swagger UI, and API responses captured
`requirements.txt & README.md`	**VERIFIED**	Complete environment spec and run documentation